Chapter 4. *Composing Objects*

So far, we've covered the low-level basics of thread safety and synchronization. But we don't want to have to analyze each memory access to ensure that our program is thread-safe; we want to be able to take thread-safe components and safely compose them into larger components or programs. This chapter covers patterns for structuring classes that can make it easier to make them thread-safe and to maintain them without accidentally undermining their safety guarantees.

4.1. Designing a Thread-safe Class

While it is possible to write a thread-safe program that stores all its state in public static fields, it is a lot harder to verify its thread safety or to modify it so that it remains thread-safe than one that uses encapsulation appropriately. Encapsulation makes it possible to determine that a class is thread-safe without having to examine the entire program.

The design process for a thread-safe class should include these three basic elements:

- Identify the variables that form the object's state;
- Identify the invariants that constrain the state variables;
- Establish a policy for managing concurrent access to the object's state.

An object's state starts with its fields. If they are all of primitive type, the fields comprise the entire state. `Counter` in **Listing 4.1** has only one field, so the `value` field comprises its entire state. The state of an object with *n* primitive fields is just the *n*-tuple of its field values; the state of a 2D `Point` is its (*x*, *y*) value. If the object has fields that are references to other objects, its state will encompass fields from the referenced objects

as well. For example, the state of a `LinkedList` includes the state of all the link node objects belonging to the list.

The *synchronization policy* defines how an object coordinates access to its state without violating its invariants or postconditions. It specifies what combination of immutability, thread confinement, and locking is used to maintain thread safety, and which variables are guarded by which locks. To ensure that the class can be analyzed and maintained, document the synchronization policy.

Listing 4.1. Simple Thread-safe Counter Using the Java Monitor Pattern.

```
@ThreadSafe
public final class Counter {
    @GuardedBy("this") private long value = 0;

    public synchronized long getValue() {
        return value;
    }
    public synchronized long increment() {
        if (value == Long.MAX_VALUE)
            throw new IllegalStateException("counter overflow");
        return ++value;
    }
}
```

4.1.1. Gathering Synchronization Requirements

Making a class thread-safe means ensuring that its invariants hold under concurrent access; this requires reasoning about its state. Objects and variables have a *state space*: the range of possible states they can take on. The smaller this state space, the easier it is to reason about. By using final fields wherever practical, you make it simpler to analyze the possible states an object can be in. (In the extreme case, immutable objects can only be in a single state.)

Many classes have invariants that identify certain states as *valid* or *invalid*. The `value` field in `Counter` is a `long`. The state space of a `long`

ranges from `Long.MIN_VALUE` to `Long.MAX_VALUE`, but `Counter` places constraints on `value`; negative values are not allowed.

Similarly, operations may have postconditions that identify certain *state transitions* as invalid. If the current state of a `Counter` is 17, the *only* valid next state is 18. When the next state is derived from the current state, the operation is necessarily a compound action. Not all operations impose state transition constraints; when updating a variable that holds the current temperature, its previous state does not affect the computation.

Constraints placed on states or state transitions by invariants and post-conditions create additional synchronization or encapsulation requirements. If certain states are invalid, then the underlying state variables must be encapsulated, otherwise client code could put the object into an invalid state. If an operation has invalid state transitions, it must be made atomic. On the other hand, if the class does not impose any such constraints, we may be able to relax encapsulation or serialization requirements to obtain greater flexibility or better performance.

A class can also have invariants that constrain multiple state variables. A number range class, like `NumberRange` in **Listing 4.10**, typically maintains state variables for the lower and upper bounds of the range. These variables must obey the constraint that the lower bound be less than or equal to the upper bound. Multivariable invariants like this one create atomicity requirements: related variables must be fetched or updated in a single atomic operation. You cannot update one, release and reacquire the lock, and then update the others, since this could involve leaving the object in an invalid state when the lock was released. When multiple variables participate in an invariant, the lock that guards them must be held for the duration of any operation that accesses the related variables.

You cannot ensure thread safety without understanding an object's invariants and postconditions. Constraints on the valid values or state transitions for state variables can create atomicity and encapsulation requirements.

4.1.2. State-dependent Operations

Class invariants and method postconditions constrain the valid states and state transitions for an object. Some objects also have methods with state-based *preconditions*. For example, you cannot remove an item from an empty queue; a queue must be in the "nonempty" state before you can remove an element. Operations with state-based preconditions are called *state-dependent* [CPJ 3].

In a single-threaded program, if a precondition does not hold, the operation has no choice but to fail. But in a concurrent program, the precondition may become true later due to the action of another thread. Concurrent programs add the possibility of waiting until the precondition becomes true, and then proceeding with the operation.

The built-in mechanisms for efficiently waiting for a condition to become true—`wait` and `notify`—are tightly bound to intrinsic locking, and can be difficult to use correctly. To create operations that wait for a precondition to become true before proceeding, it is often easier to use existing library classes, such as blocking queues or semaphores, to provide the desired state-dependent behavior. Blocking library classes such as `BlockingQueue`, `Semaphore`, and other *synchronizers* are covered in **Chapter 5**; creating state-dependent classes using the low-level mechanisms provided by the platform and class library is covered in **Chapter 14**.

4.1.3. State Ownership

We implied in **Section 4.1** that an object's state could be a subset of the fields in the object graph rooted at that object. Why might it be a subset? Under what conditions are fields reachable from a given object *not* part of that object's state?

When defining which variables form an object's state, we want to consider only the data that object *owns*. Ownership is not embodied explicitly in the language, but is instead an element of class design. If you allocate and populate a `HashMap`, you are creating multiple objects: the `HashMap` ob-

ject, a number of `Map.Entry` objects used by the implementation of `HashMap`, and perhaps other internal objects as well. The logical state of a `HashMap` includes the state of all its `Map.Entry` and internal objects, even though they are implemented as separate objects.

For better or worse, garbage collection lets us avoid thinking carefully about ownership. When passing an object to a method in C++, you have to think fairly carefully about whether you are transferring ownership, engaging in a short-term loan, or envisioning long-term joint ownership. In Java, all these same ownership models are possible, but the garbage collector reduces the cost of many of the common errors in reference sharing, enabling less-than-precise thinking about ownership.

In many cases, ownership and encapsulation go together—the object encapsulates the state it owns and owns the state it encapsulates. It is the owner of a given state variable that gets to decide on the locking protocol used to maintain the integrity of that variable's state. Ownership implies control, but once you publish a reference to a mutable object, you no longer have exclusive control; at best, you might have “shared ownership”. A class usually does not own the objects passed to its methods or constructors, unless the method is designed to explicitly transfer ownership of objects passed in (such as the synchronized collection wrapper factory methods).

Collection classes often exhibit a form of “split ownership”, in which the collection owns the state of the collection infrastructure, but client code owns the objects stored in the collection. An example is `ServletContext` from the servlet framework. `ServletContext` provides a `Map`-like object container service to servlets where they can register and retrieve application objects by name with `setAttribute` and `getAttribute`. The `ServletContext` object implemented by the servlet container must be thread-safe, because it will necessarily be accessed by multiple threads. Servlets need not use synchronization when calling `set-Attribute` and `getAttribute`, but they may have to use synchronization when *using* the objects stored in the `ServletContext`. These objects are owned by the application; they are being stored for safekeeping by the servlet container on the application's behalf. Like all

shared objects, they must be shared safely; in order to prevent interference from multiple threads accessing the same object concurrently, they should either be thread-safe, effectively immutable, or explicitly guarded by a lock.[1]

[1] Interestingly, the `HttpSession` object, which performs a similar function in the servlet framework, may have stricter requirements. Because the servlet container may access the objects in the `HttpSession` so they can be serialized for replication or passivation, they must be thread-safe because the container will be accessing them as well as the web application. (We say "may have" since replication and passivation is outside of the servlet specification but is a common feature of servlet containers.)

4.2. Instance Confinement

If an object is not thread-safe, several techniques can still let it be used safely in a multithreaded program. You can ensure that it is only accessed from a single thread (thread confinement), or that all access to it is properly guarded by a lock.

Encapsulation simplifies making classes thread-safe by promoting *instance confinement*, often just called *confinement* [CPJ 2.3.3]. When an object is encapsulated within another object, all code paths that have access to the encapsulated object are known and can be therefore be analyzed more easily than if that object were accessible to the entire program. Combining confinement with an appropriate locking discipline can ensure that otherwise non-thread-safe objects are used in a thread-safe manner.

Encapsulating data within an object confines access to the data to the object's methods, making it easier to ensure that the data is always accessed with the appropriate lock held.

Confined objects must not escape their intended scope. An object may be confined to a class instance (such as a private class member), a lexical

scope (such as a local variable), or a thread (such as an object that is passed from method to method within a thread, but not supposed to be shared across threads). Objects don't escape on their own, of course—they need help from the developer, who assists by publishing the object beyond its intended scope.

`PersonSet` in **Listing 4.2** illustrates how confinement and locking can work together to make a class thread-safe even when its component state variables are not. The state of `PersonSet` is managed by a `HashSet`, which is not thread-safe. But because `mySet` is private and not allowed to escape, the `HashSet` is confined to the `PersonSet`. The only code paths that can access `mySet` are `addPerson` and `containsPerson`, and each of these acquires the lock on the `PersonSet`. All its state is guarded by its intrinsic lock, making `PersonSet` thread-safe.

Listing 4.2. Using Confinement to Ensure Thread Safety.

```
@ThreadSafe
public class PersonSet {
    @GuardedBy("this")
    private final Set<Person> mySet = new HashSet<Person>();

    public synchronized void addPerson(Person p) {
        mySet.add(p);
    }

    public synchronized boolean containsPerson(Person p) {
        return mySet.contains(p);
    }
}
```

This example makes no assumptions about the thread-safety of `Person`, but if it is mutable, additional synchronization will be needed when accessing a `Person` retrieved from a `PersonSet`. The most reliable way to do this would be to make `Person` thread-safe; less reliable would be to guard the `Person` objects with a lock and ensure that all clients follow the protocol of acquiring the appropriate lock before accessing the `Person`.

Instance confinement is one of the easiest ways to build thread-safe classes. It also allows flexibility in the choice of locking strategy; `PersonSet` happened to use its own intrinsic lock to guard its state, but any lock, consistently used, would do just as well. Instance confinement also allows different state variables to be guarded by different locks. (For an example of a class that uses multiple lock objects to guard its state, see `ServerStatus` on 236.)

There are many examples of confinement in the platform class libraries, including some classes that exist solely to turn non-thread-safe classes into threadsafe ones. The basic collection classes such as `ArrayList` and `HashMap` are not thread-safe, but the class library provides wrapper factory methods (`Collections.synchronizedList` and friends) so they can be used safely in multithreaded environments. These factories use the Decorator pattern (**Gamma et al., 1995**) to wrap the collection with a synchronized wrapper object; the wrapper implements each method of the appropriate interface as a synchronized method that forwards the request to the underlying collection object. So long as the wrapper object holds the only reachable reference to the underlying collection (i.e., the underlying collection is confined to the wrapper), the wrapper object is then thread-safe. The Javadoc for these methods warns that all access to the underlying collection must be made through the wrapper.

Of course, it is still possible to violate confinement by publishing a supposedly confined object; if an object is intended to be confined to a specific scope, then letting it escape from that scope is a bug. Confined objects can also escape by publishing other objects such as iterators or inner class instances that may indirectly publish the confined objects.

Confinement makes it easier to build thread-safe classes because a class that confines its state can be analyzed for thread safety without having to examine the whole program.

4.2.1. The Java Monitor Pattern

Following the principle of instance confinement to its logical conclusion leads you to the *Java monitor pattern*.[2] An object following the Java monitor pattern encapsulates all its mutable state and guards it with the object's own intrinsic lock.

[2] The Java monitor pattern is inspired by Hoare's work on *monitors* (**Hoare, 1974**), though there are significant differences between this pattern and a true monitor. The bytecode instructions for entering and exiting a synchronized block are even called `monitorenter` and `monitorexit`, and Java's built-in (intrinsic) locks are sometimes called *monitor locks* or *monitors*.

`Counter` in **Listing 4.1** shows a typical example of this pattern. It encapsulates one state variable, `value`, and all access to that state variable is through the methods of `Counter`, which are all synchronized.

The Java monitor pattern is used by many library classes, such as `Vector` and `Hashtable`. Sometimes a more sophisticated synchronization policy is needed; **Chapter 11** shows how to improve scalability through finer-grained locking strategies. The primary advantage of the Java monitor pattern is its simplicity.

The Java monitor pattern is merely a convention; any lock object could be used to guard an object's state so long as it is used consistently. **Listing 4.3** illustrates a class that uses a private lock to guard its state.

Listing 4.3. Guarding State with a Private Lock.

```
public class PrivateLock {
    private final Object myLock = new Object();
    @GuardedBy("myLock") Widget widget;

    void someMethod() {
        synchronized(myLock) {
            // Access or modify the state of widget
        }
    }
}
```

There are advantages to using a private lock object instead of an object's intrinsic lock (or any other publicly accessible lock). Making the lock object private encapsulates the lock so that client code cannot acquire it, whereas a publicly accessible lock allows client code to participate in its synchronization policy—correctly or incorrectly. Clients that improperly acquire another object's lock could cause liveness problems, and verifying that a publicly accessible lock is properly used requires examining the entire program rather than a single class.

4.2.2. Example: Tracking Fleet Vehicles

`Counter` in **Listing 4.1** is a concise, but trivial, example of the Java monitor pattern. Let's build a slightly less trivial example: a “vehicle tracker” for dispatching fleet vehicles such as taxicabs, police cars, or delivery trucks. We'll build it first using the monitor pattern, and then see how to relax some of the encapsulation requirements while retaining thread safety.

Each vehicle is identified by a `String` and has a location represented by (*x*, *y*) coordinates. The `VehicleTracker` classes encapsulate the identity and locations of the known vehicles, making them well-suited as a data model in a modelview-controller GUI application where it might be shared by a view thread and multiple updater threads. The view thread would fetch the names and locations of the vehicles and render them on a display:

```
Map<String, Point> locations = vehicles.getLocations();
for (String key : locations.keySet())
    renderVehicle(key, locations.get(key));
```

Similarly, the updater threads would modify vehicle locations with data received from GPS devices or entered manually by a dispatcher through a GUI interface:

```
void vehicleMoved(VehicleMovedEvent evt) {
    Point loc = evt.getNewLocation();
    vehicles.setLocation(evt.getVehicleId(), loc.x, loc.y);
}
```

Since the view thread and the updater threads will access the data model concurrently, it must be thread-safe. **Listing 4.4** shows an implementation of the vehicle tracker using the Java monitor pattern that uses `MutablePoint` in **Listing 4.5** for representing the vehicle locations.

Even though `MutablePoint` is not thread-safe, the tracker class is. Neither the map nor any of the mutable points it contains is ever published. When we need to a return vehicle locations to callers, the appropriate values are copied using either the `MutablePoint` copy constructor or `deepCopy`, which creates a new `Map` whose values are copies of the keys and values from the old `Map`.[3]

[3] Note that `deepCopy` can't just wrap the `Map` with an `unmodifiableMap`, because that protects only the *collection* from modification; it does not prevent callers from modifying the mutable objects stored in it. For the same reason, populating the `HashMap` in `deepCopy` via a copy constructor wouldn't work either, because only the *references* to the points would be copied, not the point objects themselves.

This implementation maintains thread safety in part by copying mutable data before returning it to the client. This is usually not a performance issue, but could become one *if* the set of vehicles is very large.[4] Another consequence of copying the data on each call to `getLocation` is that the contents of the returned collection do not change even if the underlying locations change. Whether this is good or bad depends on your requirements. It could be a benefit if there are internal consistency requirements on the location set, in which case returning a consistent snapshot is critical, or a drawback if callers require up-to-date information for each vehicle and therefore need to refresh their snapshot more often.

[4] Because `deepCopy` is called from a `synchronized` method, the tracker's intrinsic lock is held for the duration of what might be a long-

running copy operation, and this could degrade the responsiveness of the user interface when many vehicles are being tracked.

4.3. Delegating Thread Safety

All but the most trivial objects are composite objects. The Java monitor pattern is useful when building classes from scratch or composing classes out of objects that are not thread-safe. But what if the components of our class are already thread-safe? Do we need to add an additional layer of thread safety? The answer is . . . "it depends". In some cases a composite made of thread-safe components is thread-safe (**Listings 4.7** and **4.9**), and in others it is merely a good start (**4.10**).

In `CountingFactorizer` on page 23, we added an `AtomicLong` to an otherwise stateless object, and the resulting composite object was still thread-safe. Since the state of `CountingFactorizer` *is* the state of the thread-safe `AtomicLong`, and since `CountingFactorizer` imposes no additional validity constraints on the state of the counter, it is easy to see that `CountingFactorizer` is thread-safe. We could say that `CountingFactorizer` *delegates* its thread safety responsibilities to the `AtomicLong`: `CountingFactorizer` is thread-safe because `AtomicLong` is.[5]

[5] If `count` were not `final`, the thread safety analysis of `CountingFactorizer` would be more complicated. If `CountingFactorizer` could modify `count` to reference a different `AtomicLong`, we would then have to ensure that this update was visible to all threads that might access the count, and that there were no race conditions regarding the value of the `count` reference. This is another good reason to use `final` fields wherever practical.

Listing 4.4. Monitor-based Vehicle Tracker Implementation.

```
@ThreadSafe
public class MonitorVehicleTracker {
    @GuardedBy("this")
    private final Map<String, MutablePoint> locations;

    public MonitorVehicleTracker(
            Map<String, MutablePoint> locations) {
        this.locations = deepCopy(locations);
    }

    public synchronized Map<String, MutablePoint> getLocations() {
        return deepCopy(locations);
    }

    public synchronized MutablePoint getLocation(String id) {
        MutablePoint loc = locations.get(id);
        return loc == null ? null : new MutablePoint(loc);
    }

    public synchronized void setLocation(String id, int x, int y) {
        MutablePoint loc = locations.get(id);
        if (loc == null)
            throw new IllegalArgumentException("No such ID: " + id);
        loc.x = x;
        loc.y = y;
    }

    private static Map<String, MutablePoint> deepCopy(
            Map<String, MutablePoint> m) {
        Map<String, MutablePoint> result =
                new HashMap<String, MutablePoint>();
        for (String id : m.keySet())
            result.put(id, new MutablePoint(m.get(id)));
        return Collections.unmodifiableMap(result);
    }
}

public class MutablePoint { /* Listing 4.5 */ }
```

Listing 4.5. Mutable Point Class Similar to `Java.awt.Point`.

```
@NotThreadSafe
public class MutablePoint {
    public int x, y;

    public MutablePoint() { x = 0; y = 0; }
    public MutablePoint(MutablePoint p) {
        this.x = p.x;
        this.y = p.y;
    }
}
```

4.3.1. Example: Vehicle Tracker Using Delegation

As a more substantial example of delegation, let's construct a version of the vehicle tracker that delegates to a thread-safe class. We store the locations in a `Map`, so we start with a thread-safe `Map` implementation, `ConcurrentHashMap`. We also store the location using an immutable `Point` class instead of `MutablePoint`, shown in **Listing 4.6**.

Listing 4.6. Immutable `Point` class used by `DelegatingVehicleTracker`.

```
@Immutable
public class Point {
    public final int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

`Point` is thread-safe because it is immutable. Immutable values can be freely shared and published, so we no longer need to copy the locations when returning them.

`DelegatingVehicleTracker` in **Listing 4.7** does not use any explicit synchronization; all access to state is managed by `ConcurrentHashMap`, and all the keys and values of the `Map` are immutable.

Listing 4.7. Delegating Thread Safety to a `ConcurrentHashMap`.

```
@ThreadSafe
public class DelegatingVehicleTracker {
    private final ConcurrentMap<String, Point> locations;
    private final Map<String, Point> unmodifiableMap;

    public DelegatingVehicleTracker(Map<String, Point> points) {
        locations = new ConcurrentHashMap<String, Point>(points);
        unmodifiableMap = Collections.unmodifiableMap(locations);
    }

    public Map<String, Point> getLocations() {
        return unmodifiableMap;
    }

    public Point getLocation(String id) {
        return locations.get(id);
    }

    public void setLocation(String id, int x, int y) {
        if (locations.replace(id, new Point(x, y)) == null)
            throw new IllegalArgumentException(
                "invalid vehicle name: " + id);
    }
}
```

If we had used the original `MutablePoint` class instead of `Point`, we would be breaking encapsulation by letting `getLocations` publish a reference to mutable state that is not thread-safe. Notice that we've changed the behavior of the vehicle tracker class slightly; while the monitor version returned a snapshot of the locations, the delegating version returns an unmodifiable but "live" view of the vehicle locations. This means that if thread *A* calls `getLocations` and thread *B* later modifies the location of some of the points, those changes are reflected in the `Map` returned to thread *A*. As we remarked earlier, this can be a benefit (more up-to-date data) or a liability (potentially inconsistent view of the fleet), depending on your requirements.

If an unchanging view of the fleet is required, `getLocations` could instead return a shallow copy of the `locations` map. Since the contents of the `Map` are immutable, only the structure of the `Map`, not the contents, must be copied, as shown in **Listing 4.8** (which returns a plain `HashMap`, since `getLocations` did not promise to return a thread-safe `Map`).

Listing 4.8. Returning a Static Copy of the Location Set Instead of a "Live" One.

```
public Map<String, Point> getLocations() {
    return Collections.unmodifiableMap(
            new HashMap<String, Point>(locations));
}
```

4.3.2. Independent State Variables

The delegation examples so far delegate to a single, thread-safe state variable. We can also delegate thread safety to more than one underlying state variable as long as those underlying state variables are *independent*, meaning that the composite class does not impose any invariants involving the multiple state variables.

`VisualComponent` in **Listing 4.9** is a graphical component that allows clients to register listeners for mouse and keystroke events. It maintains a list of registered listeners of each type, so that when an event occurs the appropriate listeners can be invoked. But there is no relationship between the set of mouse listeners and key listeners; the two are independent, and therefore `VisualComponent` can delegate its thread safety obligations to two underlying thread-safe lists.

Listing 4.9. Delegating Thread Safety to Multiple Underlying State Variables.

```
public class VisualComponent {
    private final List<KeyListener> keyListeners
        = new CopyOnWriteArrayList<KeyListener>();
    private final List<MouseListener> mouseListeners
        = new CopyOnWriteArrayList<MouseListener>();

    public void addKeyListener(KeyListener listener) {
        keyListeners.add(listener);
    }

    public void addMouseListener(MouseListener listener) {
        mouseListeners.add(listener);
    }

    public void removeKeyListener(KeyListener listener) {
        keyListeners.remove(listener);
    }

    public void removeMouseListener(MouseListener listener) {
        mouseListeners.remove(listener);
    }
}
```

`VisualComponent` uses a `CopyOnWriteArrayList` to store each listener list; this is a thread-safe `List` implementation particularly suited for managing listener lists (see **Section 5.2.3**). Each `List` is thread-safe, and because there are no constraints coupling the state of one to the state of the other, `VisualComponent` can delegate its thread safety responsibilities to the underlying `mouseListeners` and `keyListeners` objects.

4.3.3. When Delegation Fails

Most composite classes are not as simple as `VisualComponent` : they have invariants that relate their component state variables. `NumberRange` in **Listing 4.10** uses two `AtomicInteger` s to manage its state, but imposes an additional constraint—that the first number be less than or equal to the second.

Listing 4.10. Number Range Class that does Not Sufficiently Protect Its Invariants. *Don't do this.*

```
public class NumberRange {
    // INVARIANT: lower <= upper
    private final AtomicInteger lower = new AtomicInteger(0);
    private final AtomicInteger upper = new AtomicInteger(0);

    public void setLower(int i) {
        // Warning -- unsafe check-then-act
        if (i > upper.get())
            throw new IllegalArgumentException(
                    "can't set lower to " + i + " > upper");
        lower.set(i);
    }

    public void setUpper(int i) {
        // Warning -- unsafe check-then-act
        if (i < lower.get())
            throw new IllegalArgumentException(
                    "can't set upper to " + i + " < lower");
        upper.set(i);
    }

    public boolean isInRange(int i) {
        return (i >= lower.get() && i <= upper.get());
    }
}
```

`NumberRange` is not thread-safe; it does not preserve the invariant that constrains `lower` and `upper`. The `setLower` and `setUpper` methods *attempt* to respect this invariant, but do so poorly. Both `setLower` and `setUpper` are check-then-act sequences, but they do not use sufficient locking to make them atomic. If the number range holds (0, 10), and one thread calls `setLower(5)` while another thread calls `setUpper(4)`, with some unlucky timing both will pass the checks in the setters and both modifications will be applied. The result is that the range now holds (5, 4)—an invalid state. So while the underlying `AtomicInteger`s are thread-safe, the composite class is not. Because the underlying state variables `lower` and `upper` are not independent, `NumberRange` cannot simply delegate thread safety to its thread-safe state variables.

`NumberRange` could be made thread-safe by using locking to maintain its invariants, such as guarding `lower` and `upper` with a common lock. It must also avoid publishing `lower` and `upper` to prevent clients from subverting its invariants.

If a class has compound actions, as `NumberRange` does, delegation alone is again not a suitable approach for thread safety. In these cases, the class must provide its own locking to ensure that compound actions are atomic, unless the entire compound action can also be delegated to the underlying state variables.

If a class is composed of multiple *independent* thread-safe state variables and has no operations that have any invalid state transitions, then it can delegate thread safety to the underlying state variables.

The problem that prevented `NumberRange` from being thread-safe even though its state components were thread-safe is very similar to one of the rules about volatile variables described in **Section 3.1.4**: a variable is suitable for being declared `volatile` only if it does not participate in invariants involving other state variables.

4.3.4. Publishing underlying state variables

When you delegate thread safety to an object's underlying state variables, under what conditions can you publish those variables so that other classes can modify them as well? Again, the answer depends on what invariants your class imposes on those variables. While the underlying `value` field in `Counter` could take on any integer value, `Counter` constrains it to take on only positive values, and the increment operation constrains the set of valid next states given any current state. If you were to make the `value` field public, clients could change it to an invalid value, so publishing it would render the class incorrect. On the other hand, if a variable represents the current temperature or the ID of the last user to log on, then having another class modify this value at any time probably would not violate any invariants, so publishing this variable might be acceptable. (It still may not be a good idea, since publishing mutable variables constrains future development and opportunities for subclassing, but it would not *necessarily* render the class not thread-safe.)

If a state variable is thread-safe, does not participate in any invariants that constrain its value, and has no prohibited state transitions for any of its operations, then it can safely be published.

For example, it would be safe to publish `mouseListeners` or `keyListeners` in `VisualComponent`. Because `VisualComponent` does not impose any constraints on the valid states of its listener lists, these fields could be made public or otherwise published without compromising thread safety.

4.3.5. Example: Vehicle Tracker that Publishes Its State

Let's construct another version of the vehicle tracker that publishes its underlying mutable state. Again, we need to modify the interface a little bit to accommodate this change, this time using mutable but thread-safe points.

Listing 4.11. Thread-safe Mutable Point Class.

```
@ThreadSafe
public class SafePoint {
    @GuardedBy("this") private int x, y;

    private SafePoint(int[] a) { this(a[0], a[1]); }

    public SafePoint(SafePoint p) { this(p.get()); }

    public SafePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public synchronized int[] get() {
        return new int[] { x, y };
    }

    public synchronized void set(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

`SafePoint` in **Listing 4.11** provides a getter that retrieves both the *x* and *y* values at once by returning a two-element array.[6] If we provided separate getters for *x* and *y*, then the values could change between the time one coordinate is retrieved and the other, resulting in a caller seeing an inconsistent value: an (*x*, *y*) location where the vehicle never was. Using `SafePoint`, we can construct a vehicle tracker that publishes the underlying mutable state without undermining thread safety, as shown in the `PublishingVehicleTracker` class in **Listing 4.12**.

[6] The private constructor exists to avoid the race condition that would occur if the copy constructor were implemented as `this(p.x, p.y);` this is an example of the *private constructor capture idiom* (**Bloch and Gafter, 2005**).

Listing 4.12. Vehicle Tracker that Safely Publishes Underlying State.

```
@ThreadSafe
public class PublishingVehicleTracker {
    private final Map<String, SafePoint> locations;
    private final Map<String, SafePoint> unmodifiableMap;

    public PublishingVehicleTracker(
                            Map<String, SafePoint> locations) {
        this.locations
            = new ConcurrentHashMap<String, SafePoint>(locations);
        this.unmodifiableMap
            = Collections.unmodifiableMap(this.locations);
    }

    public Map<String, SafePoint> getLocations() {
        return unmodifiableMap;
    }

    public SafePoint getLocation(String id) {
        return locations.get(id);
    }

    public void setLocation(String id, int x, int y) {
        if (!locations.containsKey(id))
            throw new IllegalArgumentException(
                "invalid vehicle name: " + id);
        locations.get(id).set(x, y);
      }
}
```

`PublishingVehicleTracker` derives its thread safety from delegation to an underlying `ConcurrentHashMap`, but this time the contents of the `Map` are thread-safe mutable points rather than immutable ones. The `getLocation` method returns an unmodifiable copy of the underlying `Map`. Callers cannot add or remove vehicles, but could change the location of one of the vehicles by mutating the `SafePoint` values in the returned `Map`. Again, the "live" nature of the `Map` may be a benefit or a drawback, depending on the requirements.

`PublishingVehicleTracker` is thread-safe, but would not be so if it imposed any additional constraints on the valid values for vehicle locations. If it needed to be able to "veto" changes to vehicle locations or to take action when a location changes, the approach taken by `PublishingVehicleTracker` would not be appropriate.

4.4. Adding Functionality to Existing Thread-safe Classes

The Java class library contains many useful "building block" classes. Reusing existing classes is often preferable to creating new ones: reuse can reduce development effort, development risk (because the existing components are already tested), and maintenance cost. Sometimes a thread-safe class that supports all of the operations we want already exists, but often the best we can find is a class that supports *almost* all the operations we want, and then we need to add a new operation to it without undermining its thread safety.

As an example, let's say we need a thread-safe `List` with an atomic put-if-absent operation. The synchronized `List` implementations nearly do the job, since they provide the `contains` and `add` methods from which we can construct a put-if-absent operation.

The concept of put-if-absent is straightforward enough—check to see if an element is in the collection before adding it, and do not add it if it is already there. (Your "check-then-act" warning bells should be going off now.) The requirement that the class be thread-safe implicitly adds another requirement—that operations like put-if-absent be *atomic*. Any reasonable interpretation suggests that, if you take a `List` that does not contain object *X*, and add *X* twice with put-if-absent, the resulting collection con-

tains only one copy of *X*. But, if put-if-absent were not atomic, with some unlucky timing two threads could both see that *X* was not present and both add *X*, resulting in two copies of *X*.

The safest way to add a new atomic operation is to modify the original class to support the desired operation, but this is not always possible because you may not have access to the source code or may not be free to modify it. If you can modify the original class, you need to understand the implementation's synchronization policy so that you can enhance it in a manner consistent with its original design. Adding the new method directly to the class means that all the code that implements the synchronization policy for that class is still contained in one source file, facilitating easier comprehension and maintenance.

Another approach is to extend the class, assuming it was designed for extension. `BetterVector` in **Listing 4.13** extends `Vector` to add a `putIfAbsent` method. Extending `Vector` is straightforward enough, but not all classes expose enough of their state to subclasses to admit this approach.

Extension is more fragile than adding code directly to a class, because the implementation of the synchronization policy is now distributed over multiple, separately maintained source files. If the underlying class were to change its synchronization policy by choosing a different lock to guard its state variables, the subclass would subtly and silently break, because it no longer used the right lock to control concurrent access to the base class's state. (The synchronization policy of `Vector` is fixed by its specification, so `BetterVector` would not suffer from this problem.)

Listing 4.13. Extending `Vector` to have a Put-if-absent Method.

```
@ThreadSafe
public class BetterVector<E> extends Vector<E> {
    public synchronized boolean putIfAbsent(E x) {
        boolean absent = !contains(x);
        if (absent)
            add(x);
        return absent;
    }
}
```

4.4.1. Client-side Locking

For an `ArrayList` wrapped with a `Collections.synchronizedList` wrapper, neither of these approaches—adding a method to the original class or extending the class—works because the client code does not even know the class of the `List` object returned from the synchronized wrapper factories. A third strategy is to extend the functionality of the class without extending the class itself by placing extension code in a "helper" class.

Listing 4.14 shows a failed attempt to create a helper class with an atomic put-if-absent operation for operating on a thread-safe `List`.

Listing 4.14. Non-thread-safe Attempt to Implement Put-if-absent. *Don't do this.*

```
@NotThreadSafe
public class ListHelper<E> {
    public List<E> list =
        Collections.synchronizedList(new ArrayList<E>());
    ...
    public synchronized boolean putIfAbsent(E x) {
        boolean absent = !list.contains(x);
        if (absent)
            list.add(x);
        return absent;
    }
}
```

Why wouldn't this work? After all, `putIfAbsent` is `synchronized`, right? The problem is that it synchronizes on the *wrong lock*. Whatever lock the `List` uses to guard its state, it sure isn't the lock on the `ListHelper`. `ListHelper` provides only the *illusion of synchronization*; the various list operations, while all `synchronized`, use different locks, which means that `putIfAbsent` is *not* atomic relative to other operations on the `List`. So there is no guarantee that another thread won't modify the list while `putIfAbsent` is executing.

To make this approach work, we have to use the *same* lock that the `List` uses by using *client-side locking* or *external locking*. Client-side locking en-

tails guarding client code that uses some object *X* with the lock *X* uses to guard its own state. In order to use client-side locking, you must know what lock *X* uses.

The documentation for `Vector` and the synchronized wrapper classes states, albeit obliquely, that they support client-side locking, by using the intrinsic lock for the `Vector` or the wrapper collection (not the wrapped collection). **Listing 4.15** shows a `putIfAbsent` operation on a thread-safe `List` that correctly uses client-side locking.

Listing 4.15. Implementing Put-if-absent with Client-side Locking.

```
@ThreadSafe
public class ListHelper<E> {
    public List<E> list =
        Collections.synchronizedList(new ArrayList<E>());
    ...
    public boolean putIfAbsent(E x) {
        synchronized (list) {
            boolean absent = !list.contains(x);
            if (absent)
                list.add(x);
            return absent;
        }
    }
}
```

If extending a class to add another atomic operation is fragile because it distributes the locking code for a class over multiple classes in an object hierarchy, client-side locking is even more fragile because it entails putting locking code for class *C* into classes that are totally unrelated to *C*. Exercise care when using client-side locking on classes that do not commit to their locking strategy.

Client-side locking has a lot in common with class extension—they both couple the behavior of the derived class to the implementation of the base class. Just as extension violates encapsulation of implementation [EJ Item 14], client-side locking violates encapsulation of synchronization policy.

4.4.2. Composition

There is a less fragile alternative for adding an atomic operation to an existing class: *composition*. `ImprovedList` in **Listing 4.16** implements the `List` operations by delegating them to an underlying `List` instance, and adds an atomic `putIfAbsent` method. (Like `Collections.synchronizedList` and other collections wrappers, `ImprovedList` assumes that once a list is passed to its constructor, the client will not use the underlying list directly again, accessing it only through the `ImprovedList`.)

Listing 4.16. Implementing put-if-absent using composition.

```
@ThreadSafe
public class ImprovedList<T> implements List<T> {
    private final List<T> list;

    public ImprovedList(List<T> list) { this.list = list; }

    public synchronized boolean putIfAbsent(T x) {
        boolean contains = list.contains(x);
        if (contains)
            list.add(x);
        return !contains;
    }

    public synchronized void clear() { list.clear(); }
    // ... similarly delegate other List methods
}
```

`ImprovedList` adds an additional level of locking using its own intrinsic lock. It does not care whether the underlying `List` is thread-safe, because it provides its own consistent locking that provides thread safety even if the `List` is not thread-safe or changes its locking implementation. While the extra layer of synchronization may add some small performance penalty,[7] the implementation in `ImprovedList` is less fragile than attempting to mimic the locking strategy of another object. In effect, we've used the Java monitor pattern to encapsulate an existing `List`, and this is guaranteed to provide thread safety so long as our class holds the only outstanding reference to the underlying `List`.

[7] The penalty will be small because the synchronization on the underlying `List` is guaranteed to be uncontended and therefore fast; see

Chapter 11.

4.5. Documenting synchronization policies

Documentation is one of the most powerful (and, sadly, most underutilized) tools for managing thread safety. Users look to the documentation to find out if a class is thread-safe, and maintainers look to the documentation to understand the implementation strategy so they can maintain it without inadvertently compromising safety. Unfortunately, both of these constituencies usually find less information in the documentation than they'd like.

Document a class's thread safety guarantees for its clients; document its synchronization policy for its maintainers.

Each use of `synchronized`, `volatile`, or any thread-safe class reflects a *synchronization policy* defining a strategy for ensuring the integrity of data in the face of concurrent access. That policy is an element of your program's design, and should be documented. Of course, the best time to document design decisions is at design time. Weeks or months later, the details may be a blur—so write it down before you forget.

Crafting a synchronization policy requires a number of decisions: which variables to make `volatile`, which variables to guard with locks, which lock(s) guard which variables, which variables to make immutable or confine to a thread, which operations must be atomic, etc. Some of these are strictly implementation details and should be documented for the sake of future maintainers, but some affect the publicly observable locking behavior of your class and should be documented as part of its specification.

At the very least, document the thread safety guarantees made by a class. Is it thread-safe? Does it make callbacks with a lock held? Are there any specific locks that affect its behavior? Don't force clients to make risky guesses. If you don't want to commit to supporting client-side locking, that's fine, but say so. If you want clients to be able to create new atomic

operations on your class, as we did in **Section 4.4**, you need to document which locks they should acquire to do so safely. If you use locks to guard state, document this for future maintainers, because it's so easy—the `@GuardedBy` annotation will do the trick. If you use more subtle means to maintain thread safety, document them because they may not be obvious to maintainers.

The current state of affairs in thread safety documentation, even in the platform library classes, is not encouraging. How many times have you looked at the Javadoc for a class and wondered whether it was thread-safe?[8] Most classes don't offer any clue either way. Many official Java technology specifications, such as servlets and JDBC, woefully underdocument their thread safety promises and requirements.

[8] If you've never wondered this, we admire your optimism.

While prudence suggests that we not assume behaviors that aren't part of the specification, we have work to get done, and we are often faced with a choice of bad assumptions. Should we assume an object is thread-safe because it seems that it ought to be? Should we assume that access to an object can be made thread-safe by acquiring its lock first? (This risky technique works only if we control *all* the code that accesses that object; otherwise, it provides only the illusion of thread safety.) Neither choice is very satisfying.

To make matters worse, our intuition may often be wrong on which classes are "probably thread-safe" and which are not. As an example, `java.text.SimpleDateFormat` isn't thread-safe, but the Javadoc neglected to mention this until JDK 1.4. That this particular class isn't thread-safe comes as a surprise to many developers. How many programs mistakenly create a shared instance of a nonthread-safe object and used it from multiple threads, unaware that this might cause erroneous results under heavy load?

The problem with `SimpleDateFormat` could be avoided by not assuming a class is thread-safe if it doesn't say so. On the other hand, it is impossible to develop a servlet-based application without making some pretty questionable assumptions about the thread safety of container-provided

objects like `HttpSession`. Don't make your customers or colleagues have to make guesses like this.

4.5.1. Interpreting Vague Documentation

Many Java technology specifications are silent, or at least unforthcoming, about thread safety guarantees and requirements for interfaces such as `ServletContext`, `HttpSession`, or `DataSource`.[9] Since these interfaces are implemented by your container or database vendor, you often can't look at the code to see what it does. Besides, you don't want to rely on the implementation details of one particular JDBC driver—you want to be compliant with the standard so your code works properly with any JDBC driver. But the words “thread” and “concurrent” do not appear at all in the JDBC specification, and appear frustratingly rarely in the servlet specification. So what do you do?

[9] We find it particularly frustrating that these omissions persist despite multiple major revisions of the specifications.

You are going to have to guess. One way to improve the quality of your guess is to interpret the specification from the perspective of someone who will *implement* it (such as a container or database vendor), as opposed to someone who will merely use it. Servlets are always called from a container-managed thread, and it is safe to assume that if there is more than one such thread, the container knows this. The servlet container makes available certain objects that provide service to multiple servlets, such as `HttpSession` or `ServletContext`. So the servlet container should expect to have these objects accessed concurrently, since it has created multiple threads and called methods like `Servlet.service` from them that could reasonably be expected to access the `ServletContext`.

Since it is impossible to imagine a single-threaded context in which these objects would be useful, one has to assume that they have been made thread-safe, even though the specification does not explicitly require this. Besides, if they required client-side locking, on what lock should the client code synchronize? The documentation doesn't say, and it seems absurd to guess. This “reasonable assumption” is further bolstered by the

examples in the specification and official tutorials that show how to access `ServletContext` or `HttpSession` and do not use any client-side synchronization.

On the other hand, the objects placed in the `ServletContext` or `HttpSession` with `setAttribute` are owned by the web application, not the servlet container. The servlet specification does not suggest any mechanism for coordinating concurrent access to shared attributes. So attributes stored by the container on behalf of the web application should be thread-safe or effectively immutable. If all the container did was store these attributes on behalf of the web application, another option would be to ensure that they are consistently guarded by a lock when accessed from servlet application code. But because the container may want to serialize objects in the `HttpSession` for replication or passivation purposes, and the servlet container can't possibly know your locking protocol, you should make them thread-safe.

One can make a similar inference about the JDBC `DataSource` interface, which represents a pool of reusable database connections. A `DataSource` provides service to an application, and it doesn't make much sense in the context of a singlethreaded application. It is hard to imagine a use case that doesn't involve calling `getConnection` from multiple threads. And, as with servlets, the examples in the JDBC specification do not suggest the need for any client-side locking in the many code examples using `DataSource`. So, even though the specification doesn't promise that `DataSource` is thread-safe or require container vendors to provide a thread-safe implementation, by the same “it would be absurd if it weren't” argument, we have no choice but to assume that `DataSource.getConnection` does not require additional client-side locking.

On the other hand, we would not make the same argument about the JDBC `Connection` objects dispensed by the `DataSource`, since these are not necessarily intended to be shared by other activities until they are returned to the pool. So if an activity that obtains a JDBC `Connection` spans multiple threads, it must take responsibility for ensuring that access to the `Connection` is properly guarded by synchronization. (In

most applications, activities that use a JDBC `Connection` are implemented so as to confine the `Connection` to a specific thread anyway.)